ECE 590
Neural-Network-Based Large Language Models

Tuesdays and Thursdays 1:25-2:40
Fitzpatrick Schiciano A 1464

ECE 590
Neural-Network-Based Large Language Models

Lecture 16
03/17/2026

Basics of Q Learning for RL

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

- In basic Q learning, we assume that the states s and the actions a are each discretized, and we setup a table $Q(s, a)$ that represents the long-term expected value of taking action a in state s

- The parameter $\gamma \in [0,1]$ controls how much we discount the future, relative to future rewards

- The observed data is a sequence of states, actions, rewards, new states, *etc.*

$$\ldots, s_t,\ a_t,\ \ r_t,\ s_{t+1}, \ldots$$

Basics of Q Learning for RL

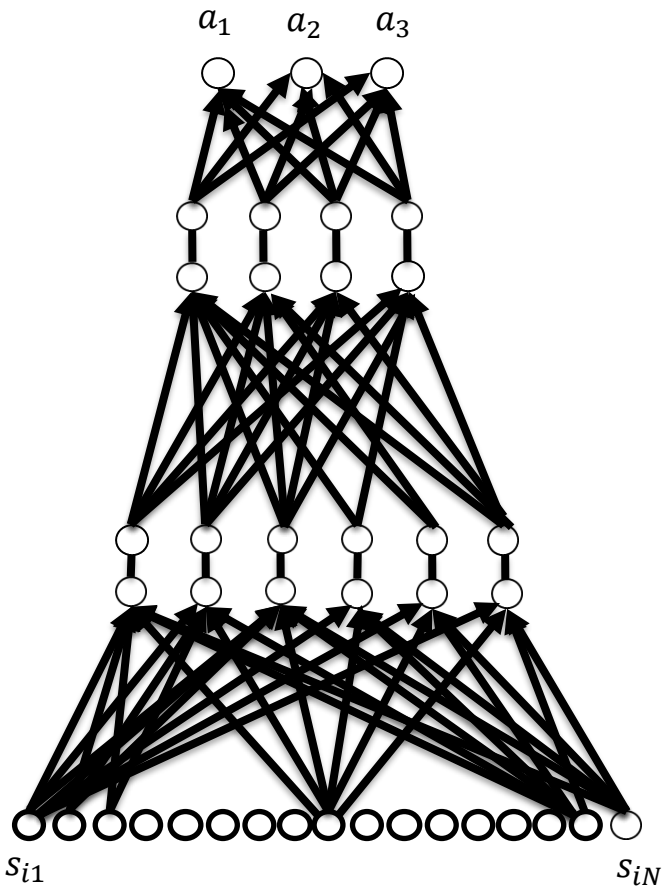
$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

- After we learn the Q function, or table, the policy is implemented as

$$\pi(s) = \text{argmax}_a \, Q(s, a)$$

- Important: In Q learning we do not explicitly model the policy, we model the Q function, and therefrom manifest the policy

Represent Q Function as a Neural Network



- The Q-function modeled via a neural network

$$U(\theta;\ s_t, a_t) = \frac{1}{2}\left[Q(s_t, a_t; \theta) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})]\ \right]^2$$

- Optimize neural network parameters θ to minimize $U(\theta; s_t, a_t)$

- States can be continuous, in this rendering actions are discrete

- Policy:

$$\pi(s) = \underset{a}{\operatorname{argmax}}\ Q(s, a; \theta)$$

Direct Learning of Policy

- In Q learning we estimate the function/table $Q_\theta(s, a)$ that quantifies the value of taking action a in state s
- The policy is implemented by selecting the most valuable action for a given state, in expectation, with long-term discounted horizon
- The policy is estimated indirectly
- In direct policy-based learning, we explicitly model the policy
- LLM is well aligned with policy learning

Direct Learning of Policy

- Let $\pi_\theta(a|s)$ represent the probability of output token sequence $a = (a_1, \dots, a_{|a|})$ where $|a|$ represents the length of generated sequence a

- The input prompt is represented by the state $s = (s_1, \dots, s_{|s|})$, which is a sequence of prompts

- The model may be implemented token by token in practice, but we wish to develop a policy that places high probability on desirable (”aligned”) entire generated sequences a

- Wish to use policy-based RL to learn the Transformer parameters θ

- Never constitute an explicit value function, directly seek to learn the policy

Desire Policy Gradient

- The key to policy-based learning is inference of an appropriate gradient

$$\theta_{k+1} = \theta_k + \alpha \nabla_\theta L\left(Data; \pi_\theta, \pi_{\theta_k}\right)_{\theta=\theta_k}$$

- The expression $L\left(Data; \pi_\theta, \pi_{\theta_k}\right)$ is a function of the data, the policy π_θ, and the old policy π_{θ_k}

- Policy refinement is manifested by gradient ascent, employing a gradient on the the function $L\left(Data; \pi_\theta, \pi_{\theta_k}\right)$

- Different policy gradient approaches consider different choices of $L\left(Data; \pi_\theta, \pi_{\theta_k}\right)$

Value Function

- Introduce a value function $V_\phi(s)$
- This represents the value (expected reward) of following our current policy $\pi_\theta(a|s)$ from state s
- We separate computation of the value function and the policy (actions), rather than combining them in Q function
- Matched with LLM learning, as $\pi_\theta(a|s)$ can be computed directly via our Transformer, as can $V_\phi(s)$

Learning of Value Function

- Assume we have a generative model represented by Transformer $\pi_\theta(a|s)$
- This Transformer represented our current policy
- Assume that for each generated token a_i we have a reward r_i
- For state (input/prompt) s we generate sequence $a_1, a_2, a_3, \ldots$
- Observe the sequence of rewards $r_1, r_2, r_3, \ldots$

Learning of Value Function

- For each state (prompt) s that we consider, manifest a set of possible outputs a, and we have a sequence of rewards for each

- For prompt s, for example generate K possible sequences $\left\{a^{(k)}\right\}_{k=1,K}$ and associated set of rewards (reflective of the quality of generation); multiple generations possibly via beam search

- For state s and generation $k \in \{1, \dots, K\}$, the discounted value of the generation for that state is

$$R = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \cdots$$

Learning of Value Function

- For state s_m assume that we have the set of long-term rewards $\left\{R_m^{(k)}\right\}_{k=1,K}$

- For each of M prompts we have K generations and cumulative discounted rewards $\left\{R_m^{(k)}\right\}_{k=1,K}$

- Use this data to constitute a model $V_\phi(s)$ which is a *value function*

- The value function is *another* Transformer, that takes s as input, and uses a linear operation acting on the vector output above the last token, to represent $V_\phi(s)$

Advantage Function

- Assume that we have the current version of the policy $\pi_\theta(a|s)$, for which we learn the associated value function $V_\phi(s)$

- We now want to update the policy parameters θ

- Using the current policy, we generate a sequence of tokens a for initial prompt s

- Define state $s_t = (s, a_1, \dots, a_t)$ which is the state input to the Transformer for generating tokens

Advantage Function

- Consider the advantage function associated with a particular sequence if text

$$A_t = -V_\phi(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t-1} r_{T-1} + \gamma^{T-t} V_\phi(s_T)$$

- Will consider $t = 0, \ldots, T-1$ where T is small compared to sequence length used to learn the value function

Advantage Function

$$A_t = -\underbrace{V_\phi(s_t)} + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\phi(s_T)$$

Value of state (prompt) based on implementing prior policy $(s, a_1, \ldots, a_t)$

Advantage Function

$$A_t = -V_\phi(s_t) + \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1}}_{\text{Observed discounted rewards of implementing refined policy for } T-t \text{ steps}} + \gamma^{T-t} V_\phi(s_T)$$

Advantage Function

$$A_t = -V_\phi(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \underbrace{\gamma^{T-t} V_\phi(s_T)}$$

Discounted value of state s_T
$(s, a_1, \ldots, a_T)$

Advantage Function

$$A_t = \underbrace{-V_\phi(s_t)}_{\text{Value of state (prompt) based on implementing prior policy}} + \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\phi(s_T)}_{\text{Empirical evaluation of current state reward}}$$

Advantage Function

$$A_t = \underbrace{-V_\phi(s_t)}_{\text{Value of state (prompt) based on implementing prior policy}} + \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\phi(s_T)}_{\text{Empirical evaluation of current state reward}}$$

- For each prompt s under consideration, we will generate many sequences, *e.g.*, via beam search
- Some sequences generated for s_t will be empirically more valuable than predicted $V_\phi(s_t)$
- For these, it is advantageous to refine the policy parameters to make such sequences more probable

Advantage Function

$$A_t = \underbrace{-V_\phi(s_t)}_{\text{Value of state (prompt) based on implementing prior policy}} + \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\phi(s_T)}_{\text{Empirical evaluation of current state reward}}$$

- If $A_t > 0$, then the refined policy is good, and we wish to encourage the associated policy
- If $A_t < 0$, then the refined policy misaligned with human desires, wish to discourage associated policy
- We generate a set of states, and associated sets of generated text, each of which has an associated measure of advantage
- Goal: Refine the policy such that advantageous sequences are probable, and disadvantageous sequences are less probable

Loss Function via Importance Sampling

$$A_t = -V_\phi(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\phi(s_T)$$

- For the current ("old") policy, we have a set of sequences and associated measure of advantage

- Let $s_t = (s, a_1, \ldots, a_t)$ have associated advantage measure A_t and $\pi_{\theta_{old}}(a_1, \ldots, a_t | s)$ reflects the probability of this sequence being generated with the old policy

- The expected advantage associated with the <u>old</u> policy is

$$\text{Old Expected Advantage} = \frac{1}{N} \sum_{n=1}^{N} A_n$$

- For notational convenience, we simply use index n to represent the nth generated example and associated measure of advantage

Loss Function via Importance Sampling

$$\text{Old Expected Advantage} = \frac{1}{N}\sum_{n=1}^{N} A_n \approx E_{\pi_{\theta_{old}}(a|s)}[A_n(a)]$$

- Here the random variable, *i.e.*, the advantage A_n, is drawn from the old policy, $\pi_{\theta_{old}}(a|s)$

- Consider the following expectation

$$\underbrace{\int da\, A(a|s)\frac{\pi_\theta(a|s)}{\pi_{\theta_{old}}(a|s)}\pi_{\theta_{old}}(a|s)}_{\text{Expected advantage under new policy } \pi_\theta} \approx \frac{1}{N}\sum_{n=1}^{N} A_n \frac{\pi_\theta(a_n|s_n)}{\pi_{\theta_{old}}(a_n|s_n)}$$

Refine Value Function & Policy

$$A_t = -V_\phi(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\phi(s_T)$$

- Want to push the policy in direction of positive Advantage

$$\hat{\theta} = \text{argmax}_\theta \frac{1}{N} \sum_{n=1}^{N} \frac{\pi_\theta(a_n|s_n)}{\pi_{\theta_{old}}(a_n|s_n)} A_n$$

- Once we update the policy, generate new sequences with it, and then update the value function model

Clipped Policy Refinement

$$A_t = -V_\phi(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\phi(s_T)$$

$$\hat{\theta} = \text{argmax}_\theta \frac{1}{N} \sum_{n=1}^{N} \frac{\pi_\theta(a_n|s_n)}{\pi_{\theta_{old}}(a_n|s_n)} A_n$$

- Define the ratio of the refined and old policy as

$$r_n(\theta) = \frac{\pi_\theta(a_n|s_n)}{\pi_{\theta_{old}}(a_n|s_n)}$$

- Constrain ("clip") how much change is allowed in the policy: $1 - \epsilon \leq r_n(\theta) \leq 1 + \epsilon$

Overall Model: Proximal Policy Optimization (PPO)

- The PPO function we wish to optimize (maximize) may be expressed as

$$L_{PPO}(\theta) = E_{q\sim p(Q), a\sim \pi_{\theta_{old}}}(S|q)\frac{1}{|a|}\sum_{t=1}^{|a|}\min\left[\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})}A_t, \text{clip}\left(\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})}, 1-\epsilon, 1+\epsilon\right)A_t\right]$$

Overall Model: Proximal Policy Optimization (PPO)

- The PPO function we wish to optimize may be expressed as

$$L_{PPO}(\theta) = E_{q\sim p(Q),a\sim\pi_{\theta_{old}}(A|q)} \frac{1}{|a|}\sum_{t=1}^{|a|} \min\left[\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})}, 1-\epsilon, 1+\epsilon\right) A_t\right]$$

Average Over a "Library" of Questions Drawn from $p(Q)$ and generated answers From Old (previous) Policy

Discourages parameters θ Moving too Far From θ_{old}

Overall Model: Proximal Policy Optimization (PPO)

- The PPO function we wish to optimize may be expressed as

$$L_{PPO}(\theta) = E_{q\sim p(Q), a\sim \pi_{\theta_{old}}(A|q)} \frac{1}{|a|} \sum_{t=1}^{|a|} \min\left[\frac{\pi_\theta(a_t|q, a_{<t})}{\pi_{\theta_{old}}(a_t|q, a_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(a_t|q, a_{<t})}{\pi_{\theta_{old}}(a_t|q, a_{<t})}, 1-\epsilon, 1+\epsilon\right) A_t\right]$$

- Let's look at a simplified form of the loss,

$$L_0(\theta) = E_{q\sim p(Q), a\sim \pi_{\theta_{old}}(A|q)} \frac{1}{|a|} \sum_{t=1}^{|a|} \frac{\pi_\theta(a_t|q, a_{<t})}{\pi_{\theta_{old}}(a_t|q, a_{<t})} A_t$$

where we ignore the clipping for ease of analysis

Proximal Policy Optimization (PPO): Weighted Cross Entropy

$$L_0(\theta) = E_{q\sim p(Q), a\sim\pi_{\theta_{old}}(A|q)} \frac{1}{|a|} \sum_{t=1}^{|a|} \frac{\pi_\theta(a_t|q, a_{<t})}{\pi_{\theta_{old}}(a_t|q, a_{<t})} A_t$$

- Notice that $\nabla_\theta \log \pi_\theta(a_t|q, a_{<t}) = \frac{\nabla_\theta \pi_\theta(a_t|q, a_{<t})}{\pi_\theta(a_t|q, a_{<t})}$ and therefore

$$\nabla_\theta \pi_\theta(a_t|q, a_{<t}) = \pi_\theta(a_t|q, a_{<t}) \nabla_\theta \log \pi_\theta(a_t|q, a_{<t})$$

Proximal Policy Optimization (PPO): Weighted Cross Entropy

$$L_0(\theta) = E_{q\sim p(Q), a\sim \pi_{\theta_{old}}(A|q)} \frac{1}{|a|} \sum_{t=1}^{|a|} \frac{\pi_\theta(a_t|q, a_{<t})}{\pi_{\theta_{old}}(a_t|q, a_{<t})} A_t$$

- Notice that $\nabla_\theta \log \pi_\theta(a_t|q, a_{<t}) = \frac{\nabla_\theta \pi_\theta(a_t|q, a_{<t})}{\pi_\theta(a_t|q, a_{<t})}$ and therefore

$$\nabla_\theta \pi_\theta(a_t|q, a_{<t}) = \pi_\theta(a_t|q, a_{<t}) \nabla_\theta \log \pi_\theta(a_t|q, a_{<t})$$

- Therefore, for learning the policy, we take the gradient

$$\nabla_\theta L_0(\theta) = E_{q\sim p(Q), a\sim \pi_{\theta_{old}}(A|q)} \frac{1}{|a|} \sum_{t=1}^{|a|} A_t \nabla_\theta \log \pi_\theta(a_t|s, a_{<t})$$

Proximal Policy Optimization (PPO): Weighted Cross Entropy

$$\nabla_\theta L_0(\theta) = E_{q\sim p(Q), a\sim \pi_{\theta_{old}}(A|q)} \frac{1}{|a|} \sum_{t=1}^{|a|} A_t \, \nabla_\theta \log \pi_\theta(a_t|s, a_{<t})$$

- Using this gradient, the policy parameters are updated as (gradient ascent, for maximization)

$$\theta = \theta_{old} + \alpha \nabla_\theta L_0(\theta)_{|\theta=\theta_{old}}$$

- Notice that

$$\nabla_\theta L_0(\theta)_{|\theta=\theta_{old}} = E_{q\sim p(Q), a\sim \pi_{\theta_{old}}(A|q)} \frac{1}{|a|} \sum_{t=1}^{|a|} A_t \, \nabla_\theta \log \pi_\theta(a_t|s, a_{<t})_{|\theta=\theta_{old}}$$

- RL may be seen as weighted cross-entropy learning, with weights A_t

PPO as Weighted Learning of Transformer Parameters

$$L_{PPO}(\theta) = E_{q\sim p(Q), a\sim \pi_{\theta_{old}}(A|q)} \frac{1}{|a|} \sum_{t=1}^{|a|} \min\left[\frac{\pi_\theta(a_t|q, a_{<t})}{\pi_{\theta_{old}}(a_t|q, a_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(a_t|q, a_{<t})}{\pi_{\theta_{old}}(a_t|q, a_{<t})}, 1-\epsilon, 1+\epsilon\right) A_t\right]$$

- Can view PPO as cross-entropy learning (like before), but now placing weights on the samples we learn on:

$$w_t = A_t$$

- Emphasize the advantageous samples, as measured by our reward model

- This is exactly what we want, with gradient steps moving in the direction that is aligned with large advantage

PPO as Weighted Learning of Transformer Parameters

$$L_{PPO}(\theta) = E_{q\sim p(Q), a\sim\pi_{\theta_{old}}(A|q)} \frac{1}{|a|}\sum_{t=1}^{|a|} \min\left[\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})}, 1-\epsilon, 1+\epsilon\right) A_t\right]$$

- Can view PPO as placing weights on the samples we learn on:

$$w_t = A_t$$

- Unlike in pretraining, we weight the importance of the samples, dependent on their alignment with preferences, as reflected in A_t

- Note: All trajectories that activate use of the Clip( ) operation via the min are not used in the gradient update, since after clipping they are independent of θ

- Final algorithm is weighted cross-entropy loss, removing from the gradient update those samples that cause too large a change in the policy, we reflected by the Clip( ) operation

Role of Clipping

$$L_{PPO}(\theta) = E_{q\sim p(Q), a\sim\pi_{\theta_{old}}(A|q)} \frac{1}{|a|}\sum_{t=1}^{|a|} \min\left[\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})}, 1-\epsilon, 1+\epsilon\right) A_t\right]$$

- Through the clipping

$$\text{clip}\left(\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})}, 1-\epsilon, 1+\epsilon\right)$$

we discourage too much change in parameters θ from θ_{old}

- Encourages gradual change in the model parameters, based on experience and reward-guided alignment

Iterative Refinement of Model Parameters

$$L_{PPO}(\theta) = E_{q\sim p(Q), a\sim\pi_{\theta_{old}}(A|q)} \frac{1}{|a|}\sum_{t=1}^{|a|} \min\left[\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})}, 1-\epsilon, 1+\epsilon\right) A_t\right]$$

- PPO is seen as weighted cross-entropy learning, with clipping used to limit change in model parameters

- We learn by performing gradient ascent

$$\theta_{k+1} = \theta_k + \alpha \nabla_\theta L_{PPO}(\theta)_{|\theta=\theta_k}$$

- After we perform gradient ascent for parameters, we then update θ_{old}, with which we then update the value function $V_\phi(s)$, generate new outcomes $a \sim \pi_{\theta_{old}}(A|q)$, and then repeat

Summary of RL for LLMs

$$L_{PPO}(\theta) = E_{q\sim p(Q),a\sim\pi_{\theta_{old}}(A|q)} \frac{1}{|a|}\sum_{t=1}^{|a|} \min\left[\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})}, 1-\epsilon, 1+\epsilon\right) A_t\right]$$

- PPO is one method of manifesting a weighted cross-entropy loss, to emphasize alignment
- There are other methods for performing weighting, which we will discuss soon
- In all such methods, we require an advantage function A_t, that defines which generated samples are most aligned with goals
- This requires a reward model, which we discuss next

Human Assessment of Responses

- Consider many different prompts, $s_1, \ldots, s_M$ where each of these is to be input to the LLM to generate an output, or action

- These $s_1, \ldots, s_M$ may represent documents to be summarized, or they may be questions for which an LLM-generated answer is desired

- For each input s_m obtain a set of possible actions/outputs, $a_{1m}, \ldots, a_{Km}$

- These K responses may be generated by different LLMs and/or by humans (*e.g.*, Reddit)

Human Evaluation

- Data: Set of LLM text inputs/prompts $\{s_m\}_{m=1,M}$ and possible responses for each $\{a_{1m}, \dots, a_{Km}\}_{m=1,M}$
- For input s_m, ask human to evaluate whether a_{im} or a_{jm} is better
- Do this with many human evaluators, for all $s_1, \dots, s_M$ and all pairs a_{im} and a_{jm}
- Now develop a model which predicts which answer humans will prefer, given input state
- This model will be used within the RL setup, to provide immediate reward evaluation for particular action

Model (1/3)

- For input s_m, assume we have asked many humans to evaluate if a_{im} or a_{jm} is better
- For reward-model design, use BERT, and use [CLS] (classification) output position
- Let $o_{im} \in R^d$ and $o_{jm} \in R^d$ represent CLS output vector for a_{im} and a_{jm}, respectively
- Learn a classifier with weight vector $w \in R^d$ applied to CLS output

Model (2/3)

- Let $o_{im} \in R^d$ and $o_{jm} \in R^d$ represent CLS output vectors for BERT inputs (s_m, a_{im}) and (s_m, a_{jm})

- Learn a classifier with weight vector $w \in R^d$ applied to CLS output

- Probability that a_{im} is preferred: $$\frac{\exp(w \cdot o_{im})}{\exp(w \cdot o_{im}) + \exp(w \cdot o_{jm})}$$

- Probability that a_{jm} is preferred: $$\frac{\exp(w \cdot o_{jm})}{\exp(w \cdot o_{im}) + \exp(w \cdot o_{jm})}$$

Model (3/3)

- Let $o_{im} \in R^d$ and $o_{jm} \in R^d$ represent CLS output vectors for a_{im} and a_{jm}

- Prob a_{im} is preferred: $\dfrac{\exp(w \cdot o_{im})}{\exp(w \cdot o_{im}) + \exp(w \cdot o_{jm})}$ Prob a_{jm} is preferred: $\dfrac{\exp(w \cdot o_{jm})}{\exp(w \cdot o_{im}) + \exp(w \cdot o_{jm})}$

- Learn w to maximize cross entropy for human-generated preferences

- Reward associated with o_{im} for input s_m: $r(s_m, a_{im}) = w \cdot o_{im}$

Other Approaches for Reward Model

- Probability that a_{im} is preferred:

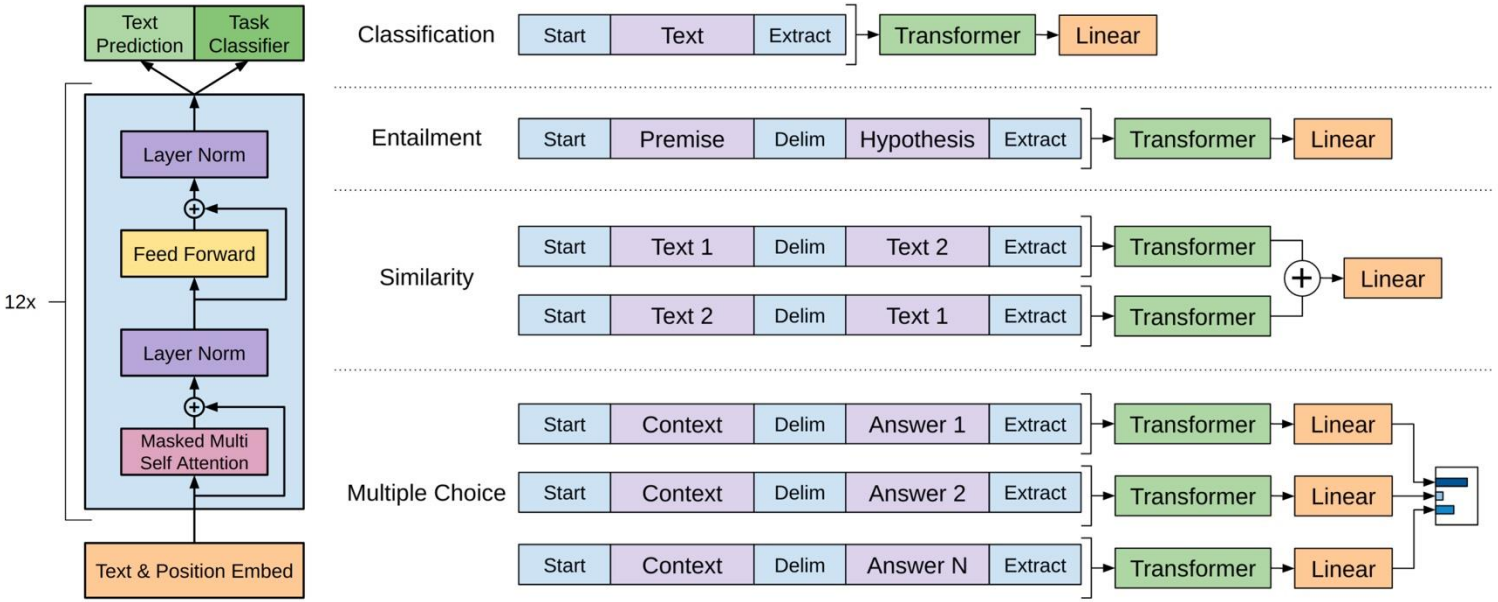
$$\frac{\exp(w \cdot o_{im})}{\exp(w \cdot o_{im}) + \exp\left(w \cdot o_{jm}\right)}$$

- Probability that a_{jm} is preferred:

$$\frac{\exp(w \cdot o_{jm})}{\exp(w \cdot o_{im}) + \exp\left(w \cdot o_{jm}\right)}$$

- We need output vectors o_{im} and o_{jm} for answers a_{im} and a_{jm} respectively to prompt m

- For the [CLS] token representations we can use the BERT model



- ❑ Can use the “Task Classifier” output, fine-tuned to the task of learning a reward model
- ❑ This is how OpenAI builds their reward model

Returning to PPO, And the Need for Sampling Methods

$$L_{PPO}(\theta) = E_{q\sim p(Q), a\sim\pi_{\theta_{old}}(A|q)} \frac{1}{|a|}\sum_{t=1}^{|a|} \min\left[\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})}A_t, \text{clip}\left(\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})}, 1-\epsilon, 1+\epsilon\right)A_t\right]$$

- One final thing to consider, which is of importance beyond RL, concerns how to sample from the policy

$$a \sim \pi_\theta(A|q)$$

- Previously we have considered beam search

- Other, better approaches for sampling from the Transformer have been developed more recently

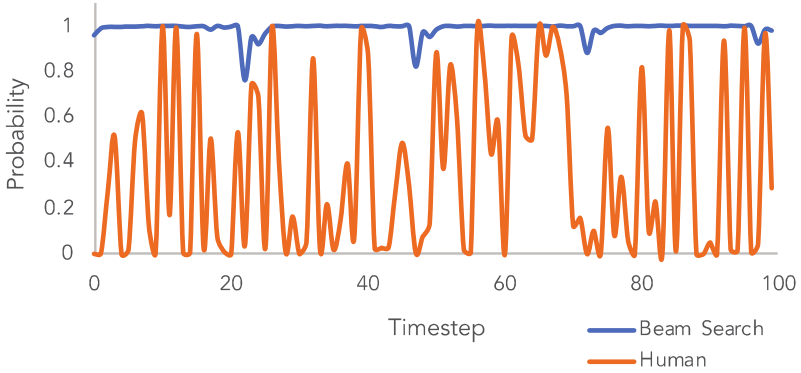
Recall Beam Search

- Given a Transformer-based policy $\pi_\theta(a|s)$, in beam search upon drawing each token, we retain B generated sequences $a_{<t} = (a_1, \dots, a_{t-1})$ where B is the beam size

- We don't take the maximum-likelihood token at each step, but we do retain the overall B most probable sequences at all times

- Is this what human-generated language looks like?

Beam Search Text is Less Surprising



Beam Search

...to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and to provide an overview of the current state-of-the-art in the field of computer vision and machine learning, and...

Human

...which grant increased life span and three years warranty. The Antec HCG series consists of five models with capacities spanning from 400W to 900W. Here we should note that we have already tested the HCG-620 in a previous review and were quite satisfied With its performance. In today's review we will rigorously test the Antec HCG-520, which as its model number implies, has 520W capacity and contrary to Antec's strong beliefs in multi-rail PSUs is equipped...

- Examination of the probability of generated tokens, as measured by an LLM
- Beam search yields text with consistently high likelihood, but it is repetitive
- Human-generated language is more random, with surprising and unexpected language

From: "The Curious Case Of Neural Text DeGeneration" by A. Holtzman, J. Buys, L. Du, M. Forbes and Y. Choi, 2020

Recall How We Generate Probabilities Over Tokens

- Using the Transformer, and auto-regressive generation:

$$p(a_1, \dots, a_n | s) = \prod_{t=1}^{n} p(a_t | s, a_{<t})$$

where $p(a_t | s, a_{<t})$ is modeled by the Transformer $\pi_\theta(a_t | s, a_{<t})$

- Assume $(s, a_{<t})$ is a sequence of N tokens, with the Nth corresponding to the last

- Let $f_N \in R^d$ be the vector output at Transformer position N, at the last layer, for input $(s, a_{<t})$

Recall How We Generate Probabilities Over Tokens

$$p(a_1, \dots, a_n | s) = \prod_{t=1}^{n} p(a_t | s, a_{<t})$$

- Let $f_N \in R^d$ be the vector output at Transformer position N, at the last layer, for input $(s, a_{<t})$

- Probability that $a_t = v \in \{1, \dots, V\}$ is

$$p(a_t = v | s, a_{<t}) = \frac{\exp(f_N^T w_v)}{\sum_{v'=1}^{V} \exp(f_N^T w_{v'})}$$

Top-k Sampling

$$p(a_t = v | s, a_{<t}) = \frac{\exp(f_N^T w_v)}{\sum_{v'=1}^{V} \exp(f_N^T w_{v'})}$$

- Select an integer k, and for each context $(s, a_{<t})$ determine those tokens $v \in \{1, \ldots, V\}$ that are among the k most probable

- For context $(s, a_{<t})$, let S be a set of the most probable k tokens

Top-k Sampling

- Consider the revised probability model for the tokens:

$$p(a_t = v | s, a_{<t}) = \frac{\exp(f_N^T w_v)\, I(v)}{\sum_{v' \in S} \exp(f_N^T w_{v'})}$$

where $I(v) = 1$ if $v \in S$ and $I(v) = 0$ otherwise

- Only the top-k tokens have non-zero probability, and the relative probability of these k tokens is defined by the relative probability in the original softmax

Top-k Sampling

- Consider the revised probability model for the tokens:

$$p(a_t = v | s, a_{<t}) = \frac{\exp(f_N^T w_v)\, I(v)}{\sum_{v' \in S} \exp(f_N^T w_{v'})}$$

where $I(v) = 1$ if $v \in S$ and $I(v) = 0$ otherwise

- To generate token a_t, we draw at random from the above refined softmax function

Drawing Multiple, Stochastically-Generated Sequences

- Using the revised probability model for the tokens:

$$p(a_t = v | s, a_{<t}) = \frac{\exp(f_N^T w_v)\, I(v)}{\sum_{v' \in S} \exp(f_N^T w_{v'})}$$

we can generated *multiple* different sequences $a_1, a_2, \ldots$

- This is like beam search, in that there are multiple sequences

- However, here each is drawn independently, using top-k stochastic sampling

Drawing Multiple, Stochastically-Generated Sequences

$$p(a_t = v | s, a_{<t}) = \frac{\exp(f_N^T w_v)\, I(v)}{\sum_{v' \in S} \exp(f_N^T w_{v'})}$$

- Given one prompt s, we may use top-k sampling to manifest multiple generated sequences

$$\left.\begin{array}{c} a_1^{(1)}, a_2^{(1)}, a_3^{(1)}, \ldots \\ a_1^{(2)}, a_2^{(2)}, a_3^{(2)}, \ldots \\ \vdots \\ a_1^{(M)}, a_2^{(M)}, a_3^{(M)}, \ldots \end{array}\right\}$$

M different stochastically Generated Sequences

Drawing Multiple, Stochastically-Generated Sequences

$$p(a_t = v | s, a_{<t}) = \frac{\exp(f_N^T w_v)\, I(v)}{\sum_{v' \in S} \exp(f_N^T w_{v'})}$$

- Given one prompt s, we may use top-k sampling to manifest multiple generated sequences

$$\left.\begin{array}{c} a_1^{(1)}, a_2^{(1)}, a_3^{(1)}, \ldots \\ a_1^{(2)}, a_2^{(2)}, a_3^{(2)}, \ldots \\ \vdots \\ a_1^{(M)}, a_2^{(M)}, a_3^{(M)}, \ldots \end{array}\right\} M \text{ different } \underline{\text{chains of thought}}$$

Second Approach to Sampling: Temperature

- Recall the original softmax-based probability over tokens:

$$p(a_t = v | s, a_{<t}) = \frac{\exp(f_N^T w_v)}{\sum_{v'=1}^{V} \exp(f_N^T w_{v'})}$$

- Introduce a "temperature" $T \in (0,1]$ and modify the probability as

$$p(a_t = v | s, a_{<t}) = \frac{\exp(f_N^T w_v / T)}{\sum_{v'=1}^{V} \exp(f_N^T w_{v'} / T)}$$

Second Approach to Sampling: Temperature

- Introduce a "temperature" $T \in (0,1]$ and modify the probability as

$$p(a_t = v | s, a_{<t}) = \frac{\exp(f_N^T w_v / T)}{\sum_{v'=1}^{V} \exp(f_N^T w_{v'} / T)}$$

- Small values of T place emphasis on the most probable tokens

- Can combine *Temperature* with subsequent top-k sampling

Third Approach to Sampling: Nucleus Sampling

- Again recall the original softmax probability over tokens

$$p(a_t = v|s, a_{<t}) = \frac{\exp(f_N^T w_v)}{\sum_{v'=1}^{V} \exp(f_N^T w_{v'})}$$

- Select a probability value $\gamma \in (0,1)$

- Determine the smallest set of tokens S such that

$$\sum_{v \in S} p(a_t = v|s, a_{<t}) > \gamma$$

Third Approach to Sampling: Nucleus Sampling

- Again recall the original softmax probability over tokens

$$p(a_t = v|s, a_{<t}) = \frac{\exp(f_N^T w_v)}{\sum_{v'=1}^{V} \exp(f_N^T w_{v'})}$$

- Determine the smallest set of tokens S such that

$$\sum_{v \in S} p(a_t = v|s, a_{<t}) > \gamma$$

- Note that the number of tokens in S will vary in size depending on the distribution of probabilities across tokens; do *not* set a fixed value of k, it varies at each step

Third Approach to Sampling: Nucleus Sampling

- With nucleus sampling we again draw tokens as

$$p(a_t = v | s, a_{<t}) = \frac{\exp(f_N^T w_v)\, I(v)}{\sum_{v' \in S} \exp(f_N^T w_{v'})}$$

but now the size of the set S changes at each step

- Again, can draw many chains of thought:

$$a_1^{(1)}, a_2^{(1)}, a_3^{(1)}, \ldots$$
$$a_1^{(2)}, a_2^{(2)}, a_3^{(2)}, \ldots$$
$$\vdots$$
$$a_1^{(M)}, a_2^{(M)}, a_3^{(M)}, \ldots$$

Back to RL

- Recall that within RL, with the "old" (prior) policy, we draw sequences, like

$$
\begin{gathered}
a_1^{(1)}, a_2^{(1)}, a_3^{(1)}, \ldots \\
a_1^{(2)}, a_2^{(2)}, a_3^{(2)}, \ldots \\
\vdots \\
a_1^{(M)}, a_2^{(M)}, a_3^{(M)}, \ldots
\end{gathered}
$$

- These can be used to compute the advantage function, assuming we have a reward model

$$A_t = -V_\phi(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\phi(s_T)$$

Back to RL

- Recall that within RL, with the "old" (prior) policy, we draw sequences, like

$$
\begin{gathered}
a_1^{(1)}, a_2^{(1)}, a_3^{(1)}, \ldots \\
a_1^{(2)}, a_2^{(2)}, a_3^{(2)}, \ldots \\
\vdots \\
a_1^{(M)}, a_2^{(M)}, a_3^{(M)}, \ldots
\end{gathered}
$$

- Will now use nucleus sampling for this

$$A_t = -V_\phi(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\phi(s_T)$$

Rethink PPO-Based RL

$$L_{PPO}(\theta) = E_{q\sim p(Q),a\sim\pi_{\theta_{old}}(A|q)} \frac{1}{|a|}\sum_{t=1}^{|a|} \min\left[\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})}, 1-\epsilon, 1+\epsilon\right) A_t\right]$$

- We have three different Transformer models:
  - The policy $\pi_\theta(a|s)$ that is responsible for generating the language
  - A Transformer used to model the value function $V_\phi(s)$ employed in the advantage model
  - A Transformer $r_\psi(a_t)$ employed to model the reward of a given action

Weirdness of Value Function for Language

- Recall the advantage function

$$A_t = -V_\phi(s_t) + r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\phi(s_T)$$

- In language, we typically do not have a reward for any one token, we get an overall reward at the end, after generating the entire sequence

- Therefore, the use of an advantage function and a value function is not fit particularly well for applying RL to language

Rethink PPO-Based RL

$$L_{PPO}(\theta) = E_{q\sim p(Q),a\sim\pi_{\theta_{old}}(A|q)} \frac{1}{|a|}\sum_{t=1}^{|a|} \min\left[\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})}, 1-\epsilon, 1+\epsilon\right) A_t\right]$$

- No longer consider a value function (reduce by one the number of needed Transformers), and draw a group of G sequences for each question q:

$$\{a_i\}_{i=1,G} \sim \pi_{\theta_{old}}(A|q)$$

- Use nucleus sampling to generate this set of G response

- Let $a_{i,t}$ represent the t-th token for the i-th sequence

Group Calculation of the Advantage Function

$$\{a_i\}_{i=1,G} \sim \pi_{\theta_{old}}(A|q)$$

- Let $r_i^{(1)}, \dots, r_i^{(K_i)}$ represent the set of rewards for sequence $i \in \{1, \dots, G\}$ up to its total length K_i

- Using all such rewards, now consider the *normalized* reward

$$\tilde{r}_i^{(k)} = \frac{r_i^{(k)} - \mu}{\sigma}$$

where μ is the mean reward and σ is the standard deviation

Group Calculation of the Advantage Function

$$\{a_i\}_{i=1,G} \sim \pi_{\theta_{old}}(A|q)$$

- Let $r_i^{(1)}, \dots, r_i^{(K_i)}$ represent the set of rewards for sequence $i \in \{1, \dots, G\}$ up to its total length K_i

$$\tilde{r}_i^{(k)} = \frac{r_i^{(k)} - \mu}{\sigma}$$

- Revised definition of the advantage:

$$\hat{A}_{i,t} = \sum_{k \geq t} \tilde{r}_i^{(k)}$$

Group Calculation of the Advantage Function

$$\{a_i\}_{i=1,G} \sim \pi_{\theta_{old}}(A|q)$$

- Let $r_i^{(1)}, \dots, r_i^{(K_i)}$ represent the set of rewards for sequence $i \in \{1, \dots, G\}$ up to its total length K_i

$$\tilde{r}_i^{(k)} = \frac{r_i^{(k)} - \mu}{\sigma}$$

- Revised definition of the advantage:

$$\hat{A}_{i,t} = \sum_{k \geq t} \tilde{r}_i^{(k)}$$

$\tilde{r}_i^{(k)}$ for $k \geq t$ reflect how advantageous token at position t is in sequence i

Rethink PPO-Based RL

$$L_{GRPO}(\theta) = E_{q\sim p(Q),\{a_i\}_{i=1,G}\sim\pi_{\theta_{old}}}(A|q)\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|a_i|}\sum_{t=1}^{|a_i|}\min\left[\frac{\pi_\theta(a_{i,t}|q,a_{i_{<t}})}{\pi_{\theta_{old}}(a_{i,t}|q,a_{i_{<t}})}\hat{A}_{i,t},\text{clip}\left(\frac{\pi_\theta(a_{i.t}|q,a_{i<t})}{\pi_{\theta_{old}}(a_{i,t}|q,a_{i<t})},1-\epsilon,1+\epsilon\right)\hat{A}_{i,t}\right]$$

- For each question $q \sim p(Q)$, average over a group of G generated sequences, manifested via nuclear sampling

Rethink PPO-Based RL

$$L_{GRPO}(\theta) = E_{q\sim p(Q),\{a_i\}_{i=1,G}\sim\pi_{\theta_{old}}}(A|q)\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|a_i|}\sum_{t=1}^{|a_i|}\min\left[\frac{\pi_\theta(a_{i,t}|q,a_{i_{<t}})}{\pi_{\theta_{old}}(a_{i,t}|q,a_{i_{<t}})}\hat{A}_{i,t},\text{clip}\left(\frac{\pi_\theta(a_{i.t}|q,a_{i<t})}{\pi_{\theta_{old}}(a_{i,t}|q,a_{i<t})},1-\epsilon,1+\epsilon\right)\hat{A}_{i,t}\right]$$

- For each question $q \sim p(Q)$, average over a group of G generated sequences, manifested via nuclear sampling

- Average over the contributions of each of the G sequences in the group

Rethink PPO-Based RL

$$L_{GRPO}(\theta) = E_{q\sim p(Q),\{a_i\}_{i=1,G}\sim\pi_{\theta_{old}}}(A|q)\,\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|a_i|}\sum_{t=1}^{|a_i|}\min\left[\frac{\pi_\theta(a_{i,t}|q,a_{i_{<t}})}{\pi_{\theta_{old}}(a_{i,t}|q,a_{i_{<t}})}\hat{A}_{i,t},\mathrm{clip}\left(\frac{\pi_\theta(a_{i.t}|q,a_{i<t})}{\pi_{\theta_{old}}(a_{i,t}|q,a_{i<t})},1-\epsilon,1+\epsilon\right)\hat{A}_{i,t}\right]$$

- For each question $q \sim p(Q)$, average over a group of G generated sequences, manifested via nuclear sampling

- Average over the contributions of each of the G sequences in the group

- For sequence $i \in \{1,\dots,G\}$, compute the average *relative* advantage based on policy $\pi_\theta(a|s)$, averaged over each step $t = 1,\dots,|a_i|$

Group Relative Policy Optimization (GRPO)

$$L_{GRPO}(\theta) = E_{q\sim p(Q),\{a_i\}_{i=1,G}\sim\pi_{\theta_{old}}(A|q)} \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|a_i|}\sum_{t=1}^{|a_i|}\min\left[\frac{\pi_\theta(a_{i,t}|q,a_{i<t})}{\pi_{\theta_{old}}(a_{i,t}|q,a_{i<t})}\hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(a_{i.t}|q,a_{i<t})}{\pi_{\theta_{old}}(a_{i,t}|q,a_{i<t})}, 1-\epsilon, 1+\epsilon\right)\hat{A}_{i,t}\right]$$

Final Piece to the RL Algorithms

$$L_{PPO}(\theta) = E_{q\sim p(Q), a\sim \pi_{\theta_{old}}(A|q)} \frac{1}{|a|}\sum_{t=1}^{|a|} \min\left[\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})}, 1-\epsilon, 1+\epsilon\right) A_t\right]$$

$$L_{GRPO}(\theta) = E_{q\sim p(Q), \{a_i\}_{i=1,G}\sim \pi_{\theta_{old}}(A|q)} \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|a_i|}\sum_{t=1}^{|a_i|} \min\left[\frac{\pi_\theta(a_{i,t}|q,a_{i<t})}{\pi_{\theta_{old}}(a_{i,t}|q,a_{i<t})} \hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(a_{i.t}|q,a_{i<t})}{\pi_{\theta_{old}}(a_{i,t}|q,a_{i<t})}, 1-\epsilon, 1+\epsilon\right) \hat{A}_{i,t}\right]$$

- In PPO and GRPO, we iterate between the prior generative policy $\pi_{\theta_{old}}(a|s)$ and the updated policy $\pi_\theta(a|s)$

- Implemented via gradient descent

$$\theta = \theta_{old} + \alpha \nabla_\theta L(\theta)_{|\theta=\theta_{old}}$$

Building on a Reference Pretrained Transformer

- We are seeking to use reinforcement learning (RL) to fine-tune a model to align with preferences, as measured by a reward function

- Typically, we start with a reference Transformer, $\pi_{ref}(a|s)$, that was pretrained on the predict-next-token task

- We want to refine the policy, yielding $\pi_\theta(a|s)$, and in practice we do not want $\pi_\theta(a|s)$ to move too far from $\pi_{ref}(a|s)$

- Wish to impose a constraint when learning via RL, that $\pi_\theta(a|s)$ is not too different from $\pi_{ref}(a|s)$

Distance Between Two Probability Distributions

- Given the context s, we have two different Transformers, $\pi_\theta(a|s)$ and $\pi_{ref}(a|s)$

- How do we measure the distance (or "divergence") between these two?

- There are many ways this can be done, a popular one is the Kullback-Liebler (KL) divergence

$$KL[\pi_\theta(a|s)||\pi_{ref}(a|s)] = \sum_a \pi_\theta(a|s) \log\left[\frac{\pi_\theta(a|s)}{\pi_{ref}(a|s)}\right]$$

$$= E_{\pi_\theta(a|s)}\left\{\log\left[\frac{\pi_\theta(a|s)}{\pi_{ref}(a|s)}\right]\right\}$$

Properties of KL Divergence

$$KL[\pi_\theta(a|s)||\pi_{ref}(a|s)] = \sum_a \pi_\theta(a|s) \log\left[\frac{\pi_\theta(a|s)}{\pi_{ref}(a|s)}\right]$$

- $KL[\pi_\theta(a|s)||\pi_{ref}(a|s)] \geq 0$
- $KL[\pi_\theta(a|s)||\pi_{ref}(a|s)] = 0$ if and only if $\pi_\theta(a|s) = \pi_{ref}(a|s)$
- $KL[\pi_\theta(a|s)||\pi_{ref}(a|s) \neq KL[\pi_{ref}(a|s)||\pi_\theta(a|s)]$

Modification to PPO

$$L_{PPO}(\theta) = E_{q\sim p(Q), a\sim \pi_{\theta_{old}}(A|q)} \frac{1}{|a|} \sum_{t=1}^{|a|} \min\left[\frac{\pi_\theta(a_t|q, a_{<t})}{\pi_{\theta_{old}}(a_t|q, a_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(a_t|q, a_{<t})}{\pi_{\theta_{old}}(a_t|q, a_{<t})}, 1-\epsilon, 1+\epsilon\right) A_t\right]$$

- Recall that we have a reward model $r_\psi(a)$ where ψ are the associated Transformer parameters

- Modify the reward model as

$$r_\psi(a) - \beta \log\left[\frac{\pi_\theta(a|s)}{\pi_{ref}(a|s)}\right]$$

Modification to PPO

$$L_{PPO}(\theta) = E_{q\sim p(Q), a\sim\pi_{\theta_{old}}(A|q)} \frac{1}{|a|}\sum_{t=1}^{|a|} \min\left[\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})}, 1-\epsilon, 1+\epsilon\right) A_t\right]$$

$$r_\psi(a) - \beta \log\left[\frac{\pi_\theta(a|s)}{\pi_{ref}(a|s)}\right]$$

- When we average over a drawn from $\pi_{old}(a|s)$ the average

$$\frac{1}{|a|}\sum_{t=1}^{|a|} \frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})} \log\left[\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{ref}(a_t|q,a_{<t})}\right] \approx KL(\pi_\theta(a|s)||\pi_{ref}(a|s))$$

Modification to PPO

$$L_{PPO}(\theta) = E_{q\sim p(Q), a\sim\pi_{\theta_{old}}(A|q)} \frac{1}{|a|}\sum_{t=1}^{|a|} \min\left[\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})}, 1-\epsilon, 1+\epsilon\right) A_t\right]$$

$$r_\psi(a) - \beta \log\left[\frac{\pi_\theta(a|s)}{\pi_{ref}(a|s)}\right]$$

- When we average over a drawn from $\pi_{old}(a|s)$ the average

$$\frac{1}{|a|}\sum_{t=1}^{|a|} \frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})} \log\left[\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{ref}(a_t|q,a_{<t})}\right] \approx KL(\pi_\theta(a|s)||\pi_{ref}(a|s))$$

Modification to PPO

$$L_{PPO}(\theta) = E_{q\sim p(Q), a\sim\pi_{\theta_{old}}(A|q)} \frac{1}{|a|}\sum_{t=1}^{|a|} \min\left[\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})}, 1-\epsilon, 1+\epsilon\right) A_t\right]$$

$$r_\psi(a) - \beta \log\left[\frac{\pi_\theta(a|s)}{\pi_{ref}(a|s)}\right]$$

- Added term to the reward penalizes (reduces reward) if the new policy is too different from the reference, pretrained model

$$\frac{1}{|a|}\sum_{t=1}^{|a|} \frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{\theta_{old}}(a_t|q,a_{<t})} \log\left[\frac{\pi_\theta(a_t|q,a_{<t})}{\pi_{ref}(a_t|q,a_{<t})}\right] \approx KL(\pi_\theta(a|s)||\pi_{ref}(a|s))$$

Modification to GRPO

- The original function we sought to maximize was

$$L_{GRPO}(\theta) = E_{q\sim p(Q),\{a_i\}_{i=1,G}\sim\pi_{\theta_{old}}}(A|q)\,\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|a_i|}\sum_{t=1}^{|a_i|}\min\left[\frac{\pi_\theta(a_{i,t}|q,a_{i<t})}{\pi_{\theta_{old}}(a_{i,t}|q,a_{i<t})}\hat{A}_{i,t},\mathrm{clip}\left(\frac{\pi_\theta(a_{i.t}|q,a_{i<t})}{\pi_{\theta_{old}}(a_{i,t}|q,a_{i<t})},1-\epsilon,1+\epsilon\right)\hat{A}_{i,t}\right]$$

- Subtract a term connected to the KL divergence:

$$L_{GRPO}(\theta) = E_{q\sim p(Q),\{a_i\}_{i=1,G}\sim\pi_{\theta_{old}}}(A|q)$$
$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|a_i|}\sum_{t=1}^{|a_i|}\min\left[\frac{\pi_\theta(a_{i,t}|q,a_{i<t})}{\pi_{\theta_{old}}(a_{i,t}|q,a_{i<t})}\hat{A}_{i,t},\mathrm{clip}\left(\frac{\pi_\theta(a_{i.t}|q,a_{i<t})}{\pi_{\theta_{old}}(a_{i,t}|q,a_{i<t})},1-\epsilon,1+\epsilon\right)\hat{A}_{i,t}\right]-\beta\cdot KL(\pi_\theta(a|s)||\pi_{ref}(a|s))$$

Modification to GRPO

Don't Move Too Far from Pre-Trained Model

$$L_{GRPO}(\theta) = E_{q\sim p(Q),\{a_i\}_{i=1,G}\sim\pi_{\theta_{old}}(A|q)}$$

$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|a_i|}\sum_{t=1}^{|a_i|}\min\left[\frac{\pi_\theta(a_{i,t}|q,a_{i<t})}{\pi_{\theta_{old}}(a_{i,t}|q,a_{i<t})}\hat{A}_{i,t},\mathrm{clip}\left(\frac{\pi_\theta(a_{i.t}|q,a_{i<t})}{\pi_{\theta_{old}}(a_{i,t}|q,a_{i<t})},1-\epsilon,1+\epsilon\right)\hat{A}_{i,t}\right]-\beta\cdot KL(\pi_\theta(a|s)||\pi_{ref}(a|s))$$

Align policy with high-reward actions